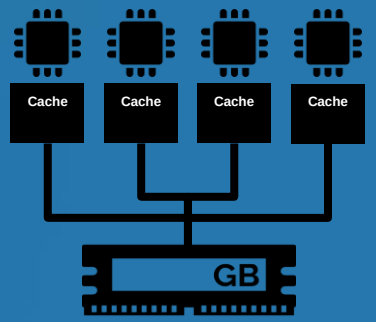
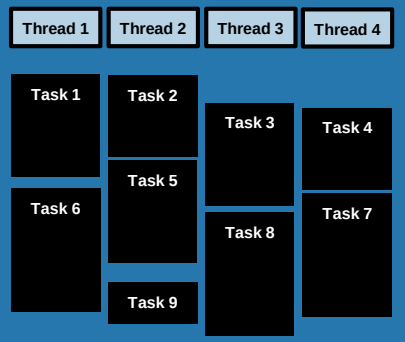
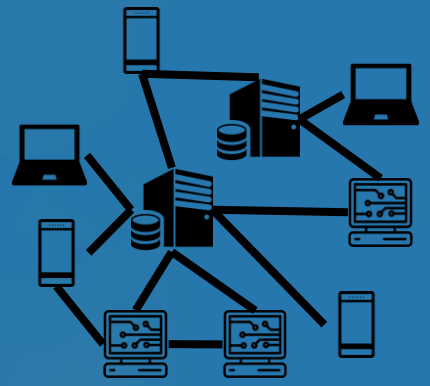
Parameterized Verification 2024

Lecture 6







Proofs of Decidability and Undecidability

To prove decidability of a decision problem, we have to show that there exists an algorithm that always produces the correct answer (and is guaranteed to terminate). Such an algorithm is called a decision procedure for this problem.

In many cases, a proof of decidability also allows to bound the complexity of the decision problem. I.e., we are not only guaranteed to get a correct answer, but for any given input we can bound the time needed to find the answer.

A decision problem is undecidable if there exists no algorithm that always computes the correct answer.

A proof of undecidability usually is a proof by contradiction, in many times by reducing a problem that is known to be undecidable to the given decision problem.

A decision problem is semi-decidable if there exists a semi-algorithm (i.e., an algorithm that is not guaranteed to terminate) that always gives the correct answer if it terminates, and is guaranteed to terminate whenever the correct answer is “yes”. Such a semi-algorithm is called a semi-decision procedure for this problem.

A Simpler Model: Two-Counter Machines

A two-counter machine (2CM) is a tuple (Q, q_0, A, δ), where

- Q is a finite set of states
- $q_0 \in Q$ is the initial state
- $q_h \in Q$ is the halting state
- $A = \{inc(c_i), dec(c_i), zero(c_i) \mid i \in \{1,2\}\}$ is the set of actions
- $\delta \subseteq Q \times A \times Q$ is the set of commands

A configuration of a 2CM is a tuple $(q, v_1, v_2) \in Q \times \mathbb{N}_0 \times \mathbb{N}_0$. It is a halting configuration if $q = q_h$.

The configuration graph of a 2CM (Q, q_0, A, δ) is the (infinite-state) transition system $(Q \times \mathbb{N}_0 \times \mathbb{N}_0, (q_0, 0,0), A, \Delta)$, where $\big((q, v_1, v_2), a, (q', v_1', v_2')\big) \in \Delta$ if $(q, a, q') \in \delta$ and

- if $a = inc(c_i)$ then $v_i' = v_i + 1$ and $v_j' = v_j$ for $j \neq i$
- if $a = dec(c_i)$ then $v_i > 0, v_i' = v_i - 1$ and $v_j' = v_j$ for $j \neq i$
- if $a = zero(c_i)$ then $v_i = v_i' = 0$ and $v_j' = v_j$ for $j \neq i$

Undecidability of Halting Problem for 2CMs

We say that a 2CM halts if a halting configuration is reachable in its configuration graph.

The halting problem for 2CMs is to decide, given a 2CM, whether it will halt.

Theorem [Minsky 1967]:

The halting problem for 2CMs is undecidable.

Minsky in fact proved undecidability of the halting problem for a slightly different machine model, also called Minsky machines. The main differences to our model are:

- our model allows non-determinism, Minsky machines are deterministic
- our model allows deadlocks (i.e., non-halting configurations without outgoing transitions), Minsky machines do not

Any Minsky machine can be translated into a 2CM, such that the Theorem above follows from Minsky's original undecidability proof.

Undecidability of the PMCP for Liveness Properties of BC Protocols

Theorem [Esparza et al. 1999]:

The PMCP for liveness properties of BC protocols is undecidable.

Proof idea:

By reduction of the halting problem for 2CMs to the PMCP.
Outline:

1. given a 2CM, define a BC protocol such that, for any fixed bound on counter values, there exists an instance such that:
   a. there is a mapping between configurations of the 2CM and states of the instance
   b. transitions in the configuration graph of the 2CM are (up to some exceptions) faithfully simulated in this instance, as long as the counter values do not exceed the bound
2. define a liveness property such that the property is violated in some instance of the BC protocol iff the 2CM can reach a halting configuration

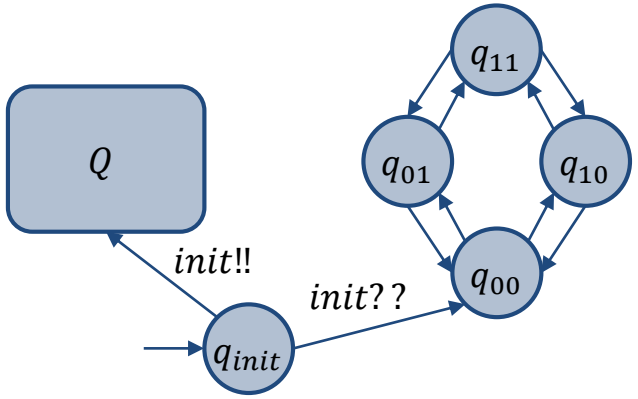
Mapping a Bounded 2CM to an Instance of a BC Protocol

A configuration (q, v_1, v_2) of a 2CM consists of

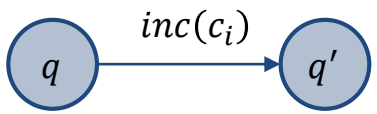
- a state $q \in Q$
- values $v_1, v_2 \in \mathbb{N}_0$ of the counters

Idea: model finite state space Q explicitly and simulate it
as the state of one process, and model the value of each counter
by a unary encoding with one Boolean variable in each other process

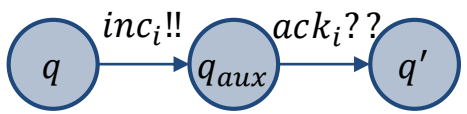
I.e., the local state space of a process roughly looks like this:
(we will see that we need some auxiliary states for the simulation)



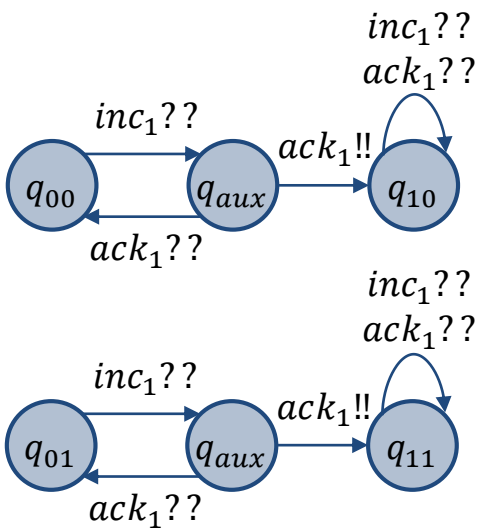
Simulating Increment and Decrement Actions



transition of the 2CM



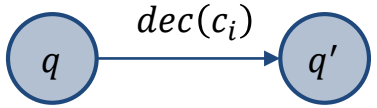
corresponding transitions of the control process

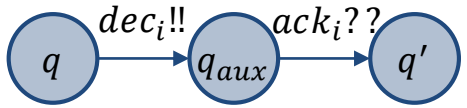


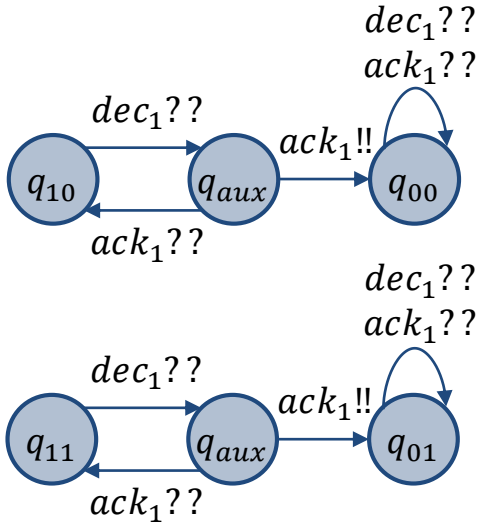
corresponding transitions of storage processes (shown for $i = 1$)

each state named q_{aux} is a fresh auxiliary state

note: can lead to deadlocks if no storage process can go to q_{aux}

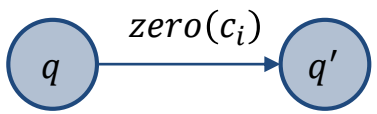




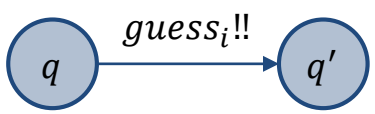


if we started from corresponding states, then after execution of the transitions in both systems, we are again in corresponding states

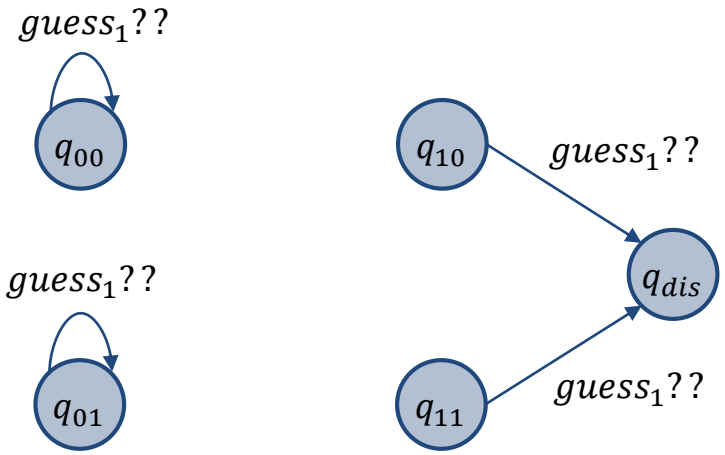
Simulating Test for Zero



transition of the 2CM



corresponding transitions of the control process



corresponding transitions of storage processes (shown for $i = 1$)

Test for zero cannot be simulated faithfully:

- that would require **all** storage processes to send a message to the control process
- but the control process can never know when it has received all messages

Therefore, we let the control process guess whether a counter is currently zero, and all processes that have a non-zero value go into state q_{dis} where they are disabled for the rest of the run.

If we started in corresponding states, then after execution of the transitions in both systems, we are either in corresponding states or at least one storage process has moved into q_{dis}.

How to Build a Proof based on Semi-Faithful Tests for Zero

Since the controller does not get feedback from storage processes in case the guess is wrong, the parameterized system instance may enter a state that corresponds to a halting configuration of the 2CM even if this configuration is not reachable in the 2CM:

$$(q_0, 0,0) \xrightarrow{inc(c_1)} (q_1, 1,0) \xrightarrow{zero(c_1)} (q_h, 1,0)$$

I.e., our simulation may reach q_h even though it is not reachable in the 2CM, but to do so it needs to "sacrifice" one or more processes.

Therefore, we can achieve the desired goal by considering

- a modified system that moves back to the initial state after reaching a state that corresponds to a halting configuration
- the liveness property

  "the system does not visit a state that corresponds to a halting configuration infinitely often"

Clarification: Undecidability Proof

Note: the simulation in the undecidability proof should not be seen as a “practical” way to reason about BC protocols or 2CMs, but rather as the simplest way to prove undecidability.

Undecidable Problem: given a 2CM, does it have a halting run?

Simulation: the 2CM has a halting run **if and only if** our parameterized system has a run that satisfies a certain property (that can be checked by parameterized model checking)

Then: the PMCP for this class of systems and properties must be undecidable.

Today: Preview

Topics:

- Guarded Protocols
- Undecidability of Safety Properties in Guarded Protocols
- Cutoffs for Disjunctive and Conjunctive Protocols

Techniques:

- Another Undecidability Proof (for Guarded Protocols)
- Cutoff Proofs (for Guarded Protocols)

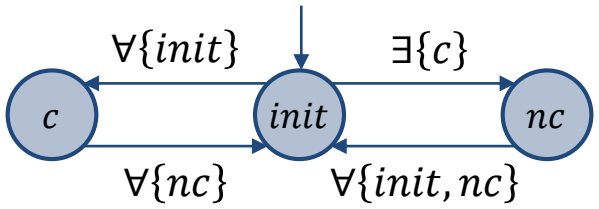
Guarded Protocols

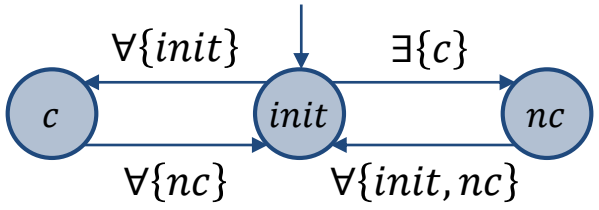
Example: Communication by Transition Guards

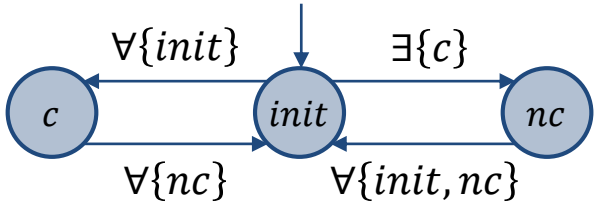
Barrier synchronization protocol:

- one process should go into critical state c
  this is only allowed if all other processes are in $init$
- if one process is in c, all others should go to nc
  going to nc is only allowed if (at least) one process is in c
- if all processes have moved out of $init$, the round concludes by all processes moving back:
  - transition $c \rightarrow init$ is only allowed if all other processes are in nc
  - transition $nc \rightarrow init$ is only allowed if all processes are in $init$ or nc

➢ the state where one process is in c and all others are in nc
acts as a barrier that synchronizes the processes
(that are otherwise completely asynchronous)







(actions omitted, all transitions are internal)

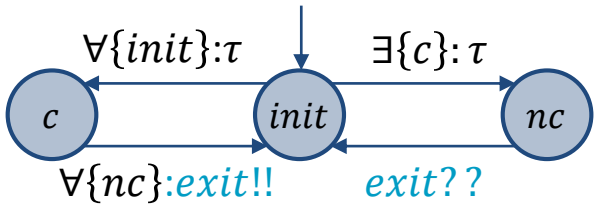
Example: Communication by Transition Guards (II)

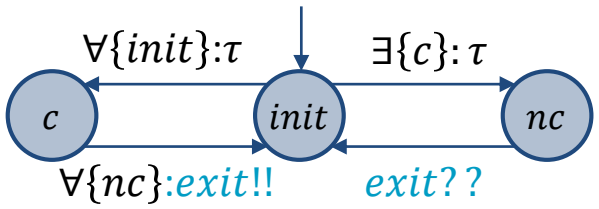
Alternative barrier synchronization protocol:

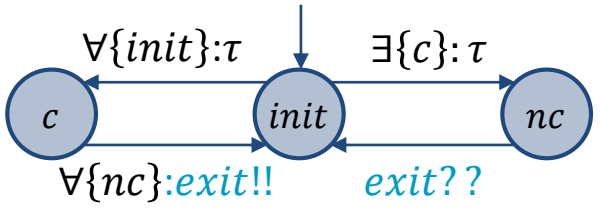
- one process should go into critical state c
  this is only allowed if all other processes are in $init$
- if one process is in c, all others should go to nc
  going to nc is only allowed if (at least) one process is in c

as before, these are internal transitions

- if we allow synchronization by broadcast, all processes can move back to $init$ in a synchronous transition:
  - transition $c \xrightarrow{exit!!} init$ is only allowed if all other processes are in nc
  - when this sending transition is taken, all processes in nc synchronize and take transition $nc \xrightarrow{exit??} init$







Guards, Guard Assignments

We say an LTS $(Q, Q_0, \Sigma, \delta, \lambda)$ is identity-labeled if $\lambda: Q \to 2^Q$ with $\lambda(q) = \{q\}$ for all $q \in Q$.

A conjunctive guard (also: universal guard) is of the form $\forall R$, where $R \subseteq Q$.

A disjunctive guard (also: existential guard) is of the form $\exists R$, where $R \subseteq Q$.

A Boolean guard is either

- a conjunctive guard
- a disjunctive guard
- $\neg g$, where g is a Boolean guard
- $f \wedge g$, where f and g are Boolean guards
- $f \vee g$, where f and g are Boolean guards

Let GD_Q be the set of all Boolean guards based on Q.

Then a guard assignment is a function $gd: \delta \to GD_Q$ that assigns to every $(q, a, q') \in \delta$ a Boolean guard g.

Evaluation of Guards

Let $C = (Q_C, Q_{C,0}, \Sigma, \delta_C, \lambda_C)$ and $U = (Q_U, Q_{U,0}, \Sigma, \delta_U, \lambda_U)$ be identity-labeled LTSs.

Guards based on $Q_C \cup Q_U$ are evaluated in global states $s \in Q_C \times (Q_U)^n$.

For $s = (q_C, q_{U1}, \dots, q_{Un})$, let $s(0)$ denote q_C and $s(i)$ denote q_{Ui} for $i \in \{1, \dots, n\}$.

Then, **for transitions of the process with index $\boldsymbol{i} \in \{\mathbf{0}, \dots, \boldsymbol{n}\}$,**

- a conjunctive guard $\forall R$ is satisfied in s iff $s(j) \in R$ for all $j \in \{0, \dots, n\} \setminus \{i\}$
- a disjunctive guard $\exists R$ is satisfied in s iff $s(j) \in R$ for at least one $j \in \{0, \dots, n\} \setminus \{i\}$
- a guard $\neg g$ is satisfied in s if guard g is not satisfied in s
- a guard $f \wedge g$ is satisfied in s if guards f and g are both satisfied in s
- a guard $f \vee g$ is satisfied in s if at least one of the guards f and g is satisfied in s

Guarded Protocols

A guarded protocol is

- a pair (C, U) of identity-labeled LTSs $C = (Q_C, Q_{C,0}, \Sigma, \delta_C, \lambda_C)$ and $U = (Q_U, Q_{U,0}, \Sigma, \delta_U, \lambda_U)$, where $\Sigma = \Sigma_{int} \cup \{out_a, in_a \mid a \in \Sigma_{sync}\}$,
- with a synchronization constraint $card$, and
- guard assignments $gd_C: \delta_C \to GD$ and $gd_U: \delta_U \to GD$, where GD is the set of all Boolean guards based on $Q_C \cup Q_U$.

Given $n \in \mathbb{N}$, an instance $C \parallel U^n$ of the guarded protocol is the identity-labeled LTS

$$(Q_c \times Q_U^n, Q_{C,0} \times Q_{U,0}^n, \Sigma_{int} \cup \Sigma_{sync}, \Delta, \lambda)$$

where $(s, a, s') \in \Delta$ if either

- (s, a, s') is an internal transition, where $s(i) \xrightarrow{a} s'(i)$ is a local transition in C or U
- (s, a, s') is a synchronous transition, where $s(i) \xrightarrow{out_a} s'(i)$ is a local transition in C or U

as defined for compositions wrt. $card$

and

the guard on the respective local transition is satisfied in s.

Reader-Writer as a Guarded Protocol

No control process needed.

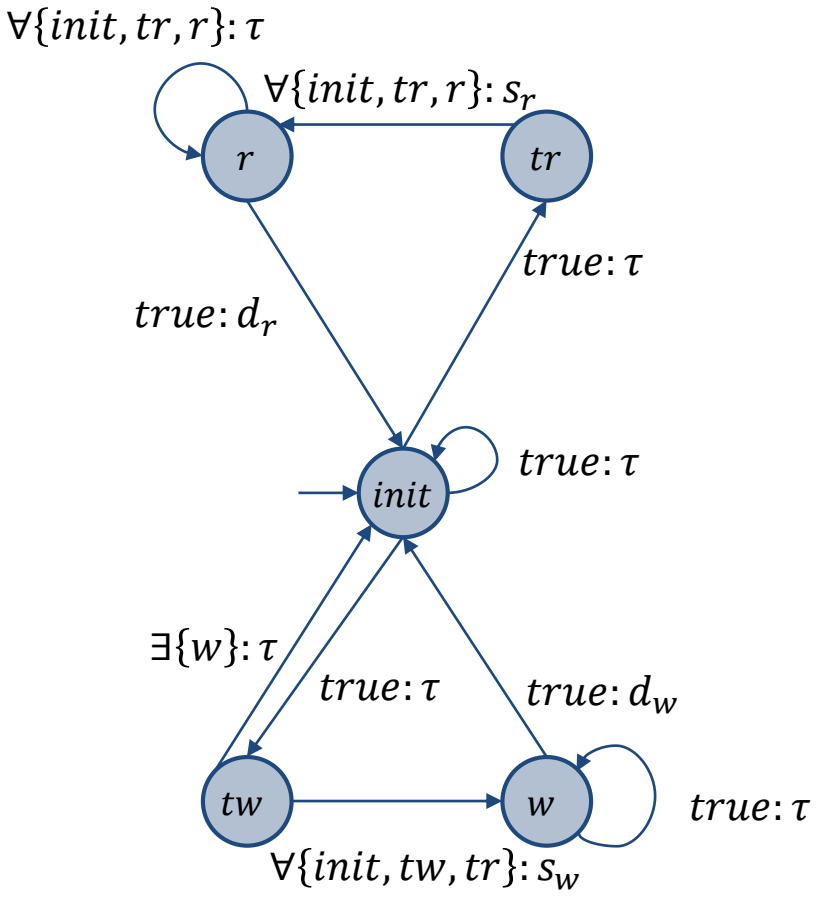
Formally, we have:

$$Q_U = \{init, tr, r, tw, w\}$$
$$Q_{U,0} = \{init\}$$
$$\Sigma_{int} = \{\tau, s_r, d_r, s_w, d_w\}$$

In picture, transitions are labeled with $g: a$, where g is the guard and a is the action.

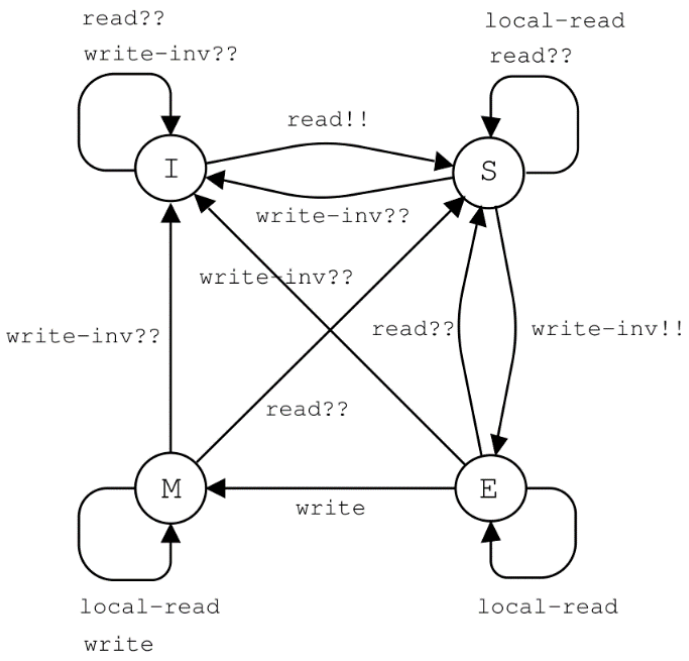
The guard $true$ is alternative notation for $\forall Q_U$.

Self-loops on r and w are optional: in this case, they define precedence of write over read – a process can enter w and stay there unconditionally even if there is another process in tr, but can enter r and stay there only if there is no process in tw.



MESI Cache Coherence Protocol as a Guarded Protocol

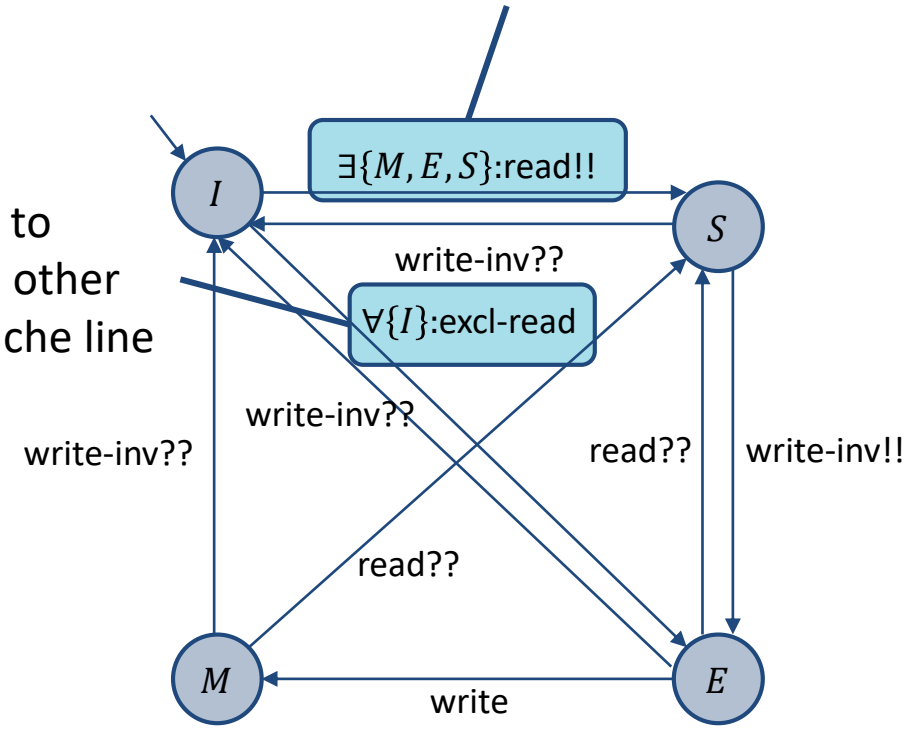
Simplified protocol,
only BC:



Full protocol,
BC+Guards:

on read, go directly to exclusive state if no other process accesses cache line

only go to shared state if cache line is shared with other process



(omitting self-loops in all states)

! With BC+Guards, most textbook cache coherence protocols can be faithfully modeled

Undecidability of PMCP for Safety Properties of Guarded Protocols

Undecidability of the PMCP for Safety Properties of Guarded Protocols

Theorem:

The PMCP for safety properties of Guarded Protocols is undecidable.

Proof idea:

By reduction of the halting problem for 2CMs to the PMCP.
Outline:

1. define a Guarded Protocol such that, for any fixed bound on counter values, there exists an instance such that:
   a. there is a mapping between configurations of the 2CM and states of the instance
   b. transitions in the configuration graph of the 2CM are faithfully simulated in this instance, as long as the counter values do not exceed the bound
2. define a safety property such that the property is violated in some instance of the Guarded Protocol iff the 2CM can reach a halting configuration

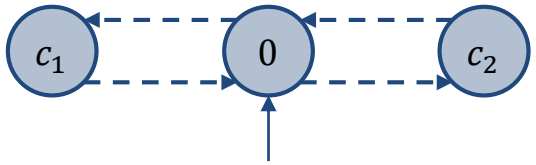
Mapping a Bounded 2CM to an Instance of a Guarded Protocol

A configuration (q, v_1, v_2) of a 2CM consists of

- a state $q \in Q$
- values $v_1, v_2 \in \mathbb{N}_0$ of the counters

Idea:
model finite state space Q explicitly in state of control process,
and model value of each counter by unary encoding in
states of storage processes

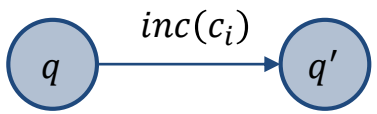
I.e., the local state space of a user process looks like this:

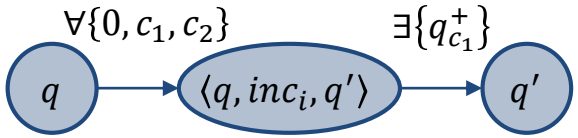


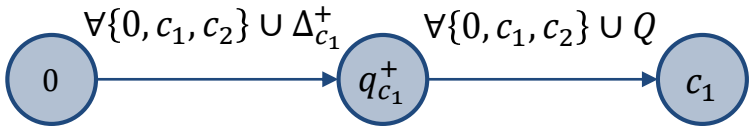
We will again need auxiliary states in both process types for simulation

Simulating Increment and Decrement Actions

$$\Delta^+_{c_1} = \{\langle q, inc_1, q'\rangle \mid q, q' \in Q\}$$







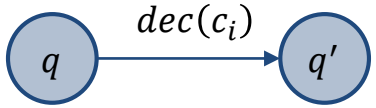
transition of the 2CM

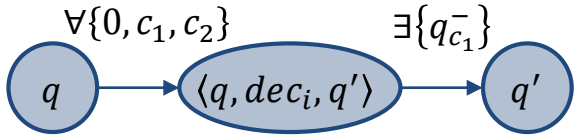
corresponding transitions of the control process

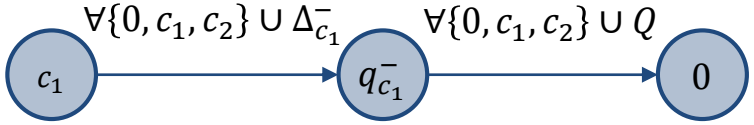
corresponding transitions of storage processes (shown for $i = 1$)

if we started from corresponding states, then after execution of the transitions in both systems, we are again in corresponding states

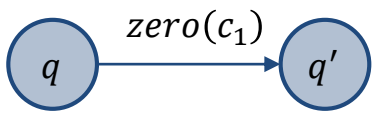
$$\Delta^-_{c_1} = \{\langle q, dec_1, q'\rangle \mid q, q' \in Q\}$$



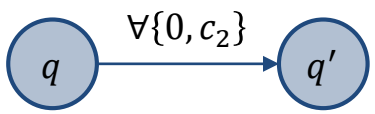




Simulating Test for Zero, Reduction to PMCP



transition of the 2CM



corresponding transitions of the control process

no transitions of storage processes needed

Reduction to PMCP:

- this construction allows us to simulate a 2CM with bound k on counters in a system instance $C \parallel U^{2k}$
- reachability of the halting state q_h in the 2CM is equivalent to reachability of state q_h by the control process in some system instance $C \parallel U^n$
- therefore, the PMCP for reachability of controller states in Guarded Protocols must be undecidable

Notes on the Undecidability Proof

Observations:

- the proof does not use synchronous transitions
  - ➢ undecidability holds for Guarded Protocols with only internal transitions
- the proof only uses disjunctive guards in two places; there is a way to remove the disjunctive guards and encode the desired behavior into a safety specification
  - ➢ undecidability holds for systems with only conjunctive guards
- with conjunctive guards we can let one process move into a distinguished part of the state space and simulate a controller
  - ➢ undecidability holds for (conjunctive) Guarded Protocols without explicit controller process

Cutoffs for Disjunctive and Conjunctive Protocols

Stutter Equivalence of Paths and Traces

Given a (finite or infinite) word w over an alphabet Σ, the de-stuttering of w is the maximal subsequence of w that does not contain a substring aa for any $a \in \Sigma$.

Informally, the de-stuttering is obtained by removing all repetitions in w.

Two words w, v are stutter equivalent if their de-stutterings are equal.

When comparing local runs in system instances of different size, we want to compare their de-stutterings, since increasing the size of the system naturally introduces additional stuttering steps. These are due to local transitions of the additional processes, and are usually not interesting for verification.

Stutter-insensitive Specifications

A set of specifications Φ is stutter-insensitive if, whenever two words (paths, traces) w, v are stutter equivalent, then $\forall \varphi \in \Phi$: $(w \vDash \varphi \Leftrightarrow v \vDash \varphi)$.

An example of stutter-insensitive specifications are temporal logic formulas in $\mathbf{LTL} \backslash \mathbf{X}$, i.e., LTL without the next-time operator $\mathbf{X}$.

Finite-state automata and Büchi automata can also be restricted such that they can only express stutter-insensitive specifications.

Proof Structure for Cutoff Theorems

In the following, we consider cutoff theorems that make a statement of the form

$$\forall C, U \in \mathcal{P}\colon \forall \varphi \in \Phi\colon \forall n \geq c\colon (C \parallel U^n \vDash \varphi \;\Leftrightarrow C \parallel U^c \vDash \varphi)$$

for a fixed $c \in \mathbb{N}$.

Note: since we assume C and U to be finite-state LTSs, the existence of a cutoff c implies that the PMCP for this class of systems and specifications is decidable.

The proofs of our cutoff theorems are split into two directions:

1. Monotonicity Lemma: $\forall n \geq c\colon (C \parallel U^n \nvDash \varphi \;\Rightarrow\; C \parallel U^{n+1} \nvDash \varphi)$
   (if there is a counterexample with n user processes, then there is also one with $n + 1$ user processes)
2. Bounding Lemma: $\forall n \geq c\colon (C \parallel U^n \nvDash \varphi \;\Rightarrow\; C \parallel U^c \nvDash \varphi)$
   (if there is a counterexample with n user processes, then there is also one with c user processes)

Cutoffs for Disjunctive Systems

We call a guarded protocol $\big((C,U), card, (gd_C, gd_U)\big)$ a disjunctive system if

- it only uses internal transitions, i.e., $\Sigma_{sync} = \emptyset$ (therefore, $card$ is irrelevant and we omit it)
- it only uses disjunctive guards

Theorem:

For any disjunctive system $\big((C,U), (gd_C, gd_U)\big)$ and any stutter-insensitive specification $\varphi(C,U)$ over local runs of C and one copy of U,

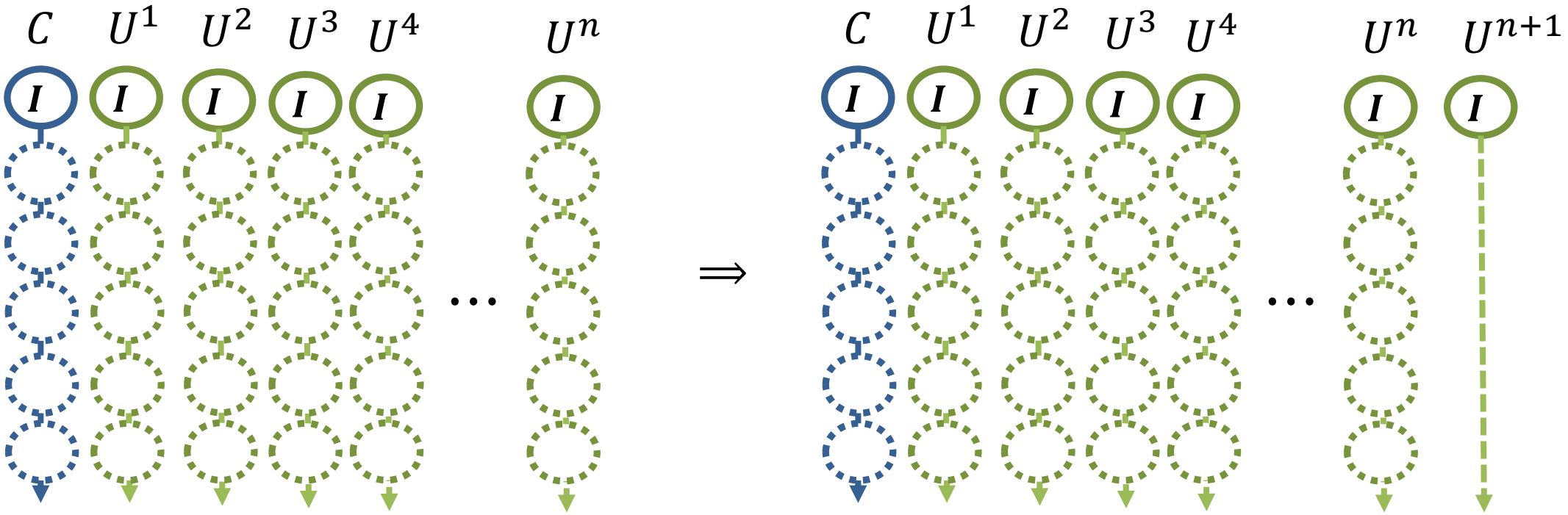
$$\forall n \geq 2|Q_U|: \Big(C \parallel U^n \vDash \varphi(C,U) \Leftrightarrow C \parallel U^{2|Q_U|} \vDash \varphi(C,U)\Big)$$

Note: in the Theorem, we have $c = 2|Q_U|$, i.e., the cutoff depends on the size of U.

Cutoffs for Disjunctive Systems: Proof

Monotonicity Lemma: $\forall n \geq 2|Q_U|$: $\left(C \parallel U^n \not\models \varphi(C,U) \;\Rightarrow\; C \parallel U^{n+1} \not\models \varphi(C,U)\right)$

Suppose we have an infinite run of $C \parallel U^n$ such that the local runs of C and one copy of U (wlog.: U^1) violate the property $\varphi(C,U)$. We construct a run of $C \parallel U^{n+1}$ such that C, U^1 also violate $\varphi(C,U)$:



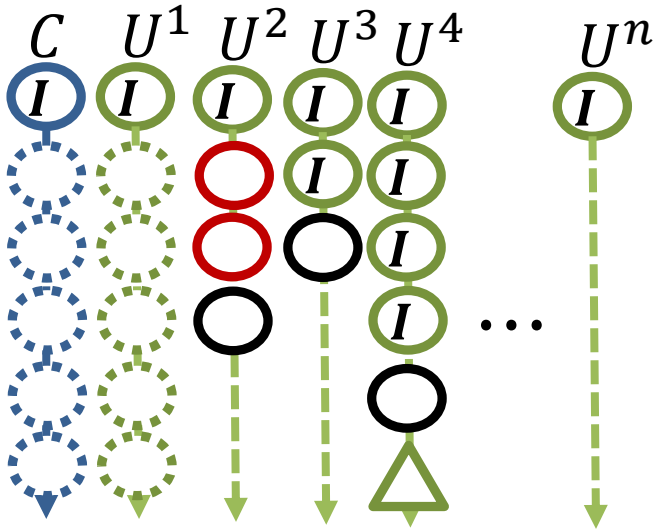
Since local runs of C, U^1 are unchanged, the property $\varphi(C,U)$ is obviously also violated in this run.

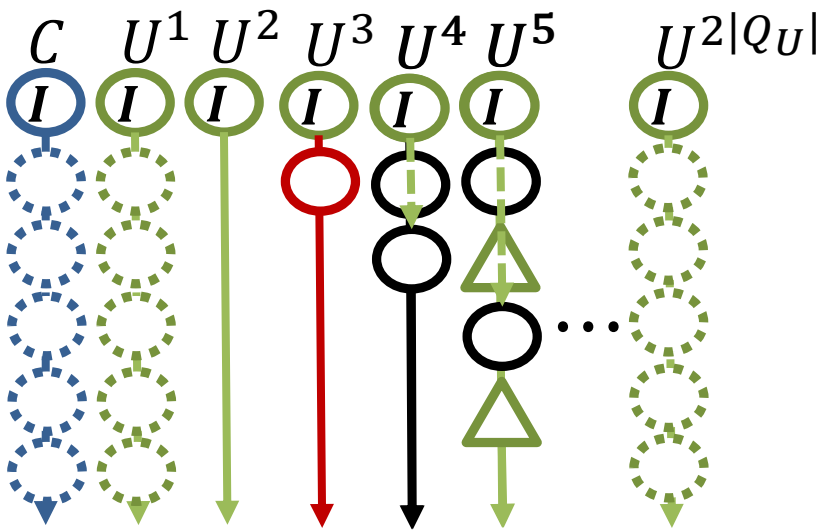
Cutoffs for Disjunctive Systems: Proof (II)

Bounding Lemma: $\forall n \geq 2|Q_U|$: $\left(C \parallel U^n \not\models \varphi(C,U) \;\Rightarrow\; C \parallel U^{2|Q_U|} \not\models \varphi(C,U)\right)$

Suppose we have an infinite run π of $C \parallel U^n$ such that the local runs of C and one copy of U (wlog.: U^1) violate the property $\varphi(C,U)$. We construct a run of $C \parallel U^{2|Q_U|}$ such that C, U^1 also violate $\varphi(C,U)$:

1. keep local runs of C and U^1
2. for every $q \in Q_U$ visited in π: add a "flooding path" for q
3. for remaining processes, just copy a local run from π; need to copy at least one infinite local run
4. split global steps that have more than one local transition, remove global stuttering (i.e., steps where no process moves)
5. to show: result is a run of $C \parallel U^{2|Q_U|}$





Cutoffs for Disjunctive Systems: Proof (III)

Bounding Lemma: $\forall n \geq 2|Q_U|$: $\left(C \parallel U^n \not\models \varphi(C,U) \;\Rightarrow\; C \parallel U^{2|Q_U|} \not\models \varphi(C,U)\right)$

To show: result is a run of $C \parallel U^{2|Q_U|}$. Since we have established interleaving semantics, i.e., at every step exactly one process makes a transition, only one thing to show: for every transition, the guard is satisfied.

From our construction it follows that at every point in time, the set of local states that are present in the current global state (from the perspective of a process that wants to move) is a superset of the states that are present at the same point in time in π. Therefore, every transition guard satisfied in π will also be satisfied here.

Note: this construction does not really need $2|Q_U|$ processes, but works also with $|Q_U| + 2$ processes.

? So why did we say that the cutoff is $2|Q_U|$?

! We assumed that π is an **infinite** run, so we have thus far not treated deadlocks!

Cutoffs for Disjunctive Systems: Proof (IV)

Bounding Lemma: $\forall n \geq 2|Q_U|$: $\left(C \parallel U^n \not\models \varphi(C,U) \;\Rightarrow\; C \parallel U^{2|Q_U|} \not\models \varphi(C,U)\right)$

Suppose we have a deadlocked run π of $C \parallel U^n$. We construct a deadlocked run of $C \parallel U^{2|Q_U|}$:

Let s be the final state of π. For every $q \in Q_U$ that appears in s, we distinguish whether a transition from q would be enabled if there was another process in q (i.e., whether $q \in R$ for any transition out of q guarded by $\exists R$). Let $dead_1$ be the set of local states where this is the case, and $dead_2$ the states where it is not.

To obtain a deadlocked run, first copy directly the local runs of C and those that end in $dead_1$.

Then use the same construction as before to add flooding paths for every state q that is visited in π by one of the processes that we did not copy. We let them stay in their respective state q until the state of C and the local states in $dead_1$ have been reached. Then, if $q \in dead_2$, it can stay there, and otherwise we continue the local run until it reaches a state in $dead_2$ (this is possible because this is what it did in π).

Observations:

- this construction needs $2|Q_U|$ copies of U in the worst case (with closer analysis: $2|Q_U| - 1$)
- it ensures that all transition guards are satisfied and the final state is deadlocked

Cutoffs for Disjunctive Systems: Discussion

Observations:

- the construction for infinite runs in the bounding lemma does not preserve fairness; in fact, it explicitly violates fairness!
  - ➢ there are also cutoffs that preserve fairness (based on a more involved construction)
- cutoffs that grow with the size of the component LTSs will in many applications be not very useful, because even with cutoff the complexity is too big
  - ➢ cutoffs that depend on other properties of the system can be found (e.g., instead of $|Q_U|$ we can take the number of all states that appear in at least one guard)
  - ➢ smaller cutoffs may be possible under additional restrictions/assumptions on the system

Tight Cutoffs for Guarded Protocols with Fairness

Simon Außerlechner[1], Swen Jacobs[2], Ayrat Khalimov[1]

[1] IAIK, Graz University of Technology, Austria
[2] Reactive Systems Group, Saarland University, Germany

Analyzing Guarded Protocols: Better Cutoffs, More Systems, More Expressivity

Swen Jacobs[1], Mouhammad Sakr[1]

[1] Reactive Systems Group, Saarland University, Germany

Cutoffs for Init-Conjunctive Systems

We call a guarded protocol $\big((C,U), card, (gd_C, gd_U)\big)$ an init-conjunctive system if

- it only uses internal transitions, i.e., $\Sigma_{sync} = \emptyset$
- it only uses conjunctive guards
- initial states cannot cause violation of a guard, i.e.,
  for every guard $\forall R$ that is assigned by gd_C or gd_U, we have $Q_{C,0} \subseteq R$ and $Q_{U,0} \subseteq R$

Theorem:

For any init-conjunctive system $\big((C,U), card, (gd_C, gd_U)\big)$ and any stutter-insensitive specification $\varphi(C,U)$ over local runs of C and one copy of U,

$$\forall n \geq 2|Q_U|: \Big(C \parallel U^n \vDash \varphi(C,U) \;\Leftrightarrow C \parallel U^{2|Q_U|} \vDash \varphi(C,U)\Big)$$

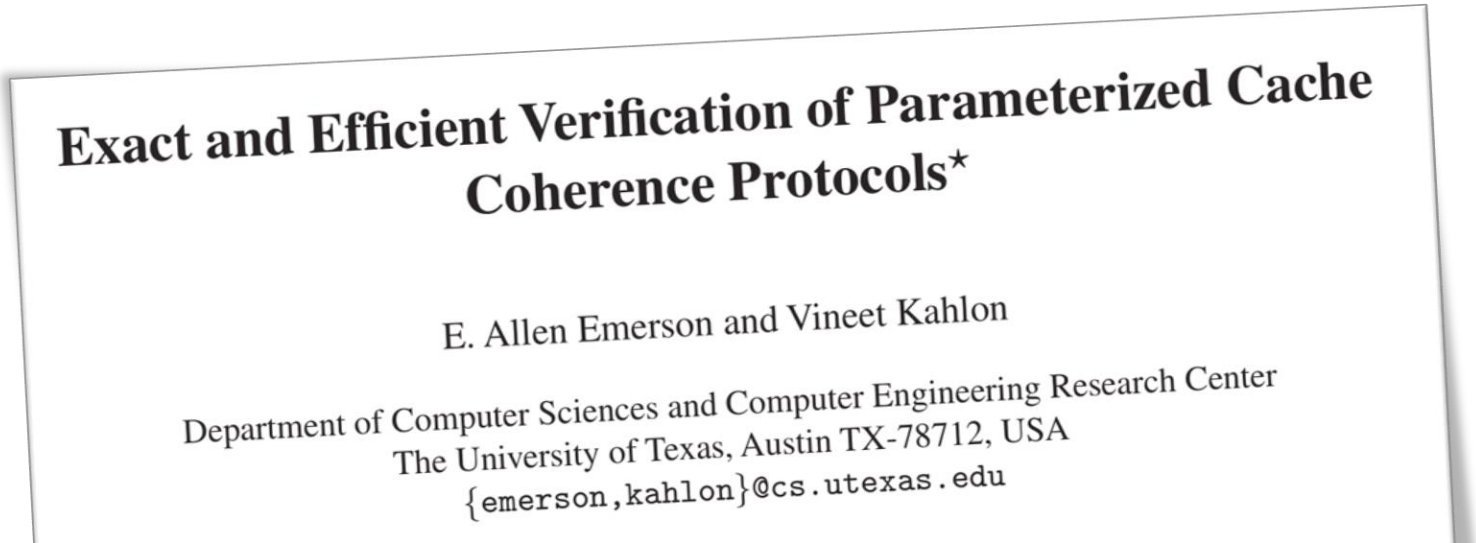
(Monotonicity Lemma and Bounding Lemma for infinite runs are easy, deadlocks are the interesting case)

Cutoffs for Cache Coherence Protocols

In a similar way, we can obtain cutoffs for a class of Guarded Protocols that allows us to model a wide range of cache coherence protocols.

However, these cutoffs are rather specialized to this application area:

- they only allow one conjunctive guard $\forall Q_{C,0} \cup Q_{U,0}$
- they only allow one disjunctive guard $\exists (Q_C \cup Q_U) \setminus \{Q_{C,0}, Q_{U,0}\}$
- they have additional restrictions on the transitions



Exact and Efficient Verification of Parameterized Cache Coherence Protocols*

E. Allen Emerson and Vineet Kahlon

Department of Computer Sciences and Computer Engineering Research Center
The University of Texas, Austin TX-78712, USA
{emerson,kahlon}@cs.utexas.edu

Today: Summary

- Guarded Protocols (very strong model, with guards and synchronization)
- Undecidability of PMCP for Safety Properties of Guarded Protocols (how to simulate a 2CM)
- Cutoffs for Disjunctive and Conjunctive Protocols (decidability for restricted classes)

Organizational Matters:

1. There will be no lecture and no tutorials next week
2. Remarks/Questions about the oral exam